2025 年第四届“创新杯”大学生
大数据挑战赛论文

题　　目 基于随机森林与 BP 神经网络的工业设备传感器故障判断与寿命分析

摘　要

随着中国经济由高速增长阶段转向高质量发展阶段，我国正处于发展方式转变、经济结构优化、增长动力转换的关键攻关期。我国作为制造业大国，仍存在过度依赖劳动力、资源浪费、安全防护措施尚不完善等问题。因此，制造业迫切需要“智能化转型”。得益于人工智能、大数据科学的发展，人们有能力利用大数据和 AI 算法实时监控设备状态、物料消耗和生产进度，动态调整生产计划，减少资源浪费（如能源、原材料）。神经网络作为一类有代表性的机器学习算法，提供了一种有效的非线性预测方法。本文将聚焦于工业设备传感器故障与寿命，利用**随机森林**模型与 **BP 神经网络**对机床运行情况展开分析。

针对问题 A，根据核心传感器的数据集，将数据划分为数值类型数据、布尔类型数据、枚举类型数据。分别对不同的数据类型做特定的处理。在第一问中，所针对的目标为七日之内发生故障的概率，通过查看数据分布，发现数据存在不平衡问题，故采取过采样（SMTOE）方法，对数据集进行扩充。采用网格搜索方法，确定随机森林的合适超参数，利用所给输入数据等关键变量来进行模型拟合。得到随机森林模型，该模型的**准确率、召回率、F1 值分别为 0.9998、0.9983、0.9991**。从随机森林导出的重要度中，**运行小时数、振动、温度、噪音与功率**是最重要的五个特征。并结合 SHAP 分析，分析问题。

针对问题 B，本题数据预处理方法、相关性分析及显著性分析同问题 A。为了增强预测的稳定性和精确度，本文采用**带有残差连接的 BP 神经网络**。通过引入残差连接，使得网络能够在训练过程中更好地传播梯度，从而提高模型性能。对于隐藏层的神经元个数，使用遗传算法优化获取。所得到的 BP 神经网络模型结构为 26-64-64-1，该模型在 80%的数据集（320000 条）下训练，在 20%数据集数据（80000 条）测试。所得 **MSE 为 50.83，R^2 为 0.9714，最重要的三个特征为温度、声音、震动**。并结合 SHAP 分析，分析问题。

关键词 ：随机森林；残差神经网络；SMTOE；SHAP 分析

目录

一、问题重述 1
1.1 问题背景 1
1.2 问题重述 1
二、问题分析 1
2.1 问题 A 的分析 1
2.2 问题 B 的分析 2
三、问题假设 2
四、符号说明 3
五、模型建立与求解 3
5.1 问题 A 的建立与求解 3
5.1.1 数据预处理 3
5.1.2 数据相关性分析 5
5.1.3 GridSearchCV（网格搜索交叉验证） 7
5.1.4 随机森林分类模型 7
5.1.5 模型评估 8
5.2 问题 B 的建立与求解 9
5.2.1 数据预处理 9
5.2.2 数据相关性分析 10
5.2.2 定义数据加载器 10
5.2.3 残差神经网络结构 10
5.2.4 利用遗传算法搜寻最佳网络层数 11
5.2.5 训练神经网络 12
5.2.6 模型评估 12
六、模型分析 14
6.1 问题 A 的模型分析 14
6.1.1 随机森林导出的特征重要度 14
6.1.2 基于随机森林的 SHAP 值分析 14
6.2 问题 B 的模型分析 18
七、模型的评价 20
6.1 模型优点 20
6.2 模型缺点 20

一、问题重述

1.1 问题背景

为了应对传统制造业中普遍存在的生产模式粗放、管理效率低下、资源消耗过高、安全保障不足以及过度依赖低技能劳动力等问题，同时在全球新一轮科技革命和产业变革浪潮下，“智能制造”、“工业 4.0”等理念蓬勃兴起，智慧工厂应运而生并深度融入制造领域。它旨在通过数字化、智能化和自动化技术的深度融合，从根本上重塑制造业的生产模式、管理流程和价值链，以推动工业向更高级形态、更可持续的方向发展，最终实现降本增效、安全可控和绿色发展的目标。新质生产力在制造业的体现，正聚焦于这种创新驱动、数据赋能、柔性生产、资源优化和人机协同的深刻转型。智慧工厂不仅代表了当前制造业升级的核心路径，也为构建未来竞争力提供了关键支撑。

1.2 问题重述

根据某工厂传感器模拟器数据集，对所给数据进行分析以及查阅相关资料，建立数学模型解决以下问题：

问题 A：针对机床设备未来七天是否会发生故障问题，输入机器类型、运行小时数、温度、振动、声音、油位、冷却液位、功耗、距上次维护天数、维护历史次数、故障历史次数、人工智能监控、过去 30 天错误编码等特征，构建分类模型，预测机床设备未来七天是否会发生故障，并分析最重要的五个特征。

问题 B：针对机床设备的剩余使用寿命问题，输入运行小时数、温度、振动、油位、冷却液位、维护历史次数、故障历史次数等特征，构建回归模型，预测机床设备的剩余使用寿命。并分析特征重要性。

二、问题分析

2.1 问题 A 的分析

本文首先对工厂传感器模拟器数据进行数据清洗，接着对数据绘制箱线图，进行异常值检测，去除异常值。将特征分为数值型、布尔型、枚举型，三种类型的数据，对枚

举类型进行编号、对数值类型数据进行归一化、对布尔类型数据进行 0-1 赋值，添加缺失指示器。使用箱线图查看各特征数值分布，使用 KDE 图查看数据分布是否合理，并利用 Person 相关对各个特征搜寻自变量与因变量之间的关系。将数据集按 80%-20%的数据分为训练集、测试集。根据分析建立随机森林分类模型，使用训练好的随机森林模型预测测试集中的机床设备七日内故障概率。并使用 SHAP 分析，分析最重要的五个特征。

2.2 问题 B 的分析

本文对机床设备剩余使用寿命预测所使用的的数据预处理方法与问题 A 一致，进行数据清洗，对数据绘制箱线图，进行异常值检测，去除异常值。对枚举类型进行编号、对数值类型数据进行归一化、对布尔类型数据进行 0-1 赋值，添加缺失指示器。处理后得到用于建立模型的数据集。再利用 Person 相关对各特征进行相关性分析和显著性分析，探寻自变量与因变量之间的关系。将数据集按 80%-20%的数据分为训练集、测试集。根据分析建立神经网络回归模型。使用训练好的神经网络模型对测试集中机床设备剩余使用寿命进行预测。并使用 SHAP 分析，分析最重要的三个特征。

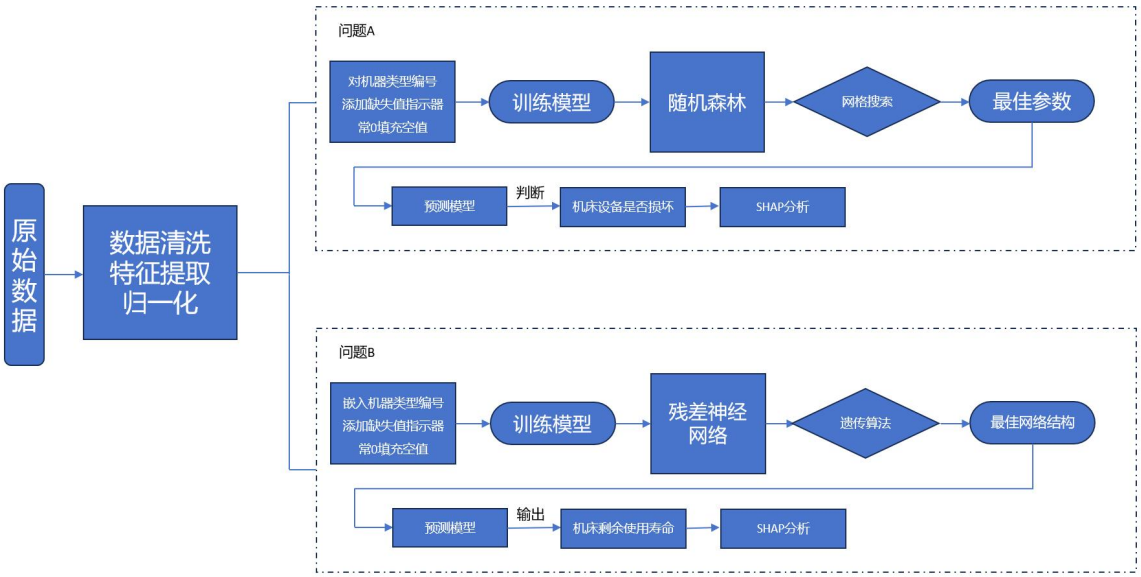

图 1 问题研究思路

三、问题假设

（1）假设脱敏和数据变换处理对模型无影响；

（2）假设不考虑指标之外的因素影响；

四、符号说明

符号	说明
r	皮尔逊相关系数
R^2	拟合优度
MSE	均方误差

五、模型建立与求解

5.1 问题 A 的建立与求解

5.1.1 数据预处理

（1）特征分类

首先根据特征的特点将特征分为数值型、枚举型、布尔型，其中“机器编号”与“安装年份”因为与机器损坏并无关系，所以在建立模型的时候，不考虑这两个特征。

表 1 特征类型与处理方式

符号	输入参数	处理方式
数值型	Operational_Hours （运行小时数）	归一化
	Temperature_C （温度）	
	Vibration_mms （振动）	
	Sound_dB （声音）	
	Oil_Level_pct （油位百分比）	
	Coolant_Level_pct （冷却液位百分比）	
	Power_Consumption_kW （功耗）	
	Last_Maintenance_Days_Ago （距上次维护天数）	
	Maintenance_History_Count （历史维护次数）	
	Failure_History_Count （故障历史次数）	

	Remaining_Useful_Life_days （剩余使用寿命）	
	Laser_Intensity （激光强度）	
	Hydraulic_Pressure_bar （液压压力）	
	Coolant_Flow_L_min （冷却液流量）	
	Heat_Index （热指数）	
布尔型	AI_Supervision （人工智能监控）	0-1 化
	Failure_Within_7_Days （7 天内故障预测）	
枚举型	Error_Codes_Last_30_Days （过去 30 天错误编码）	编码
	AI_Override_Events （人工智能覆盖率）	
	Machine_Type （机器类型）	

（2）数据分布与数据处理

使用箱线图查看 11 个数值类型的数据分布图。我们发现，数据都处在较为合理的区间，虽然有部分特征的数据落在了上四分位数和下四分位数之外，但考虑到故障可能会导致数据出现异常值，所以，对落在四分位数外的数据都做了保留。

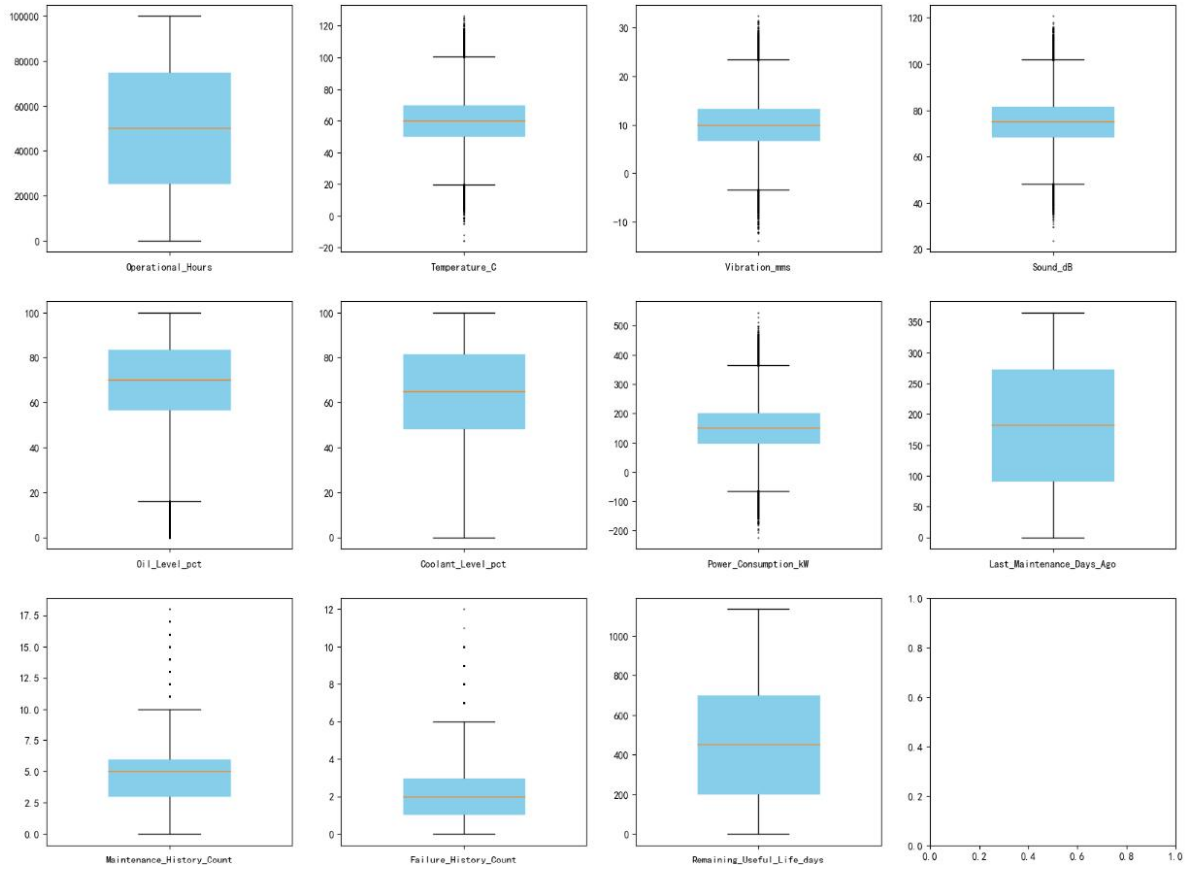

图 2 数据分布箱线图

对 15 个数值类型的变量采用归一化处理方式，注意到温度、振动、功率出现了负数，可能有两种情况。第一种情况是机器可能出现故障导致，这些特征出现异常值，第二种情况是，机器正常工作的环境需要低温，正常工作振动幅度本身为负，机器有做负功的情况。无论哪种情况，负值都是可接受的，因此不对负值做去除操作。此外，因为温度的正负、功率的正负、振动的正负，都具有明显的物理分割意义。所以将数据放缩到-1，1 范围。虽然随机森林是基于决策树的，因此对数据是否归一化不做要求，但考虑到 B 问中的神经网络的梯度传播需要归一化，因此对数据进行统一归一化。

对布尔类型数据采用 0-1 化处理方式。

对枚举类型采用编码的方式处理，查看数据集，发现一共有 33 个机器类型，依次对它们赋予一个数字。

将数据集的 80%作为训练集，20%作为测试集

（3）特征工程

对于 Laser_Intensity（激光强度）、Hydraulic_Pressure_bar（液压压力）、Coolant_Flow_L_min（冷却液流量）、Heat_Index（热指数）特征是特定类别的机械才有的特征，因此存在大量的缺失值，但可能会对模型预测有帮助，因此，选择保留这四个特征，空值用 0 填充，为了区别空值，额外增加四个特征：Laser_Intensity_missing、Hydraulic_Pressure_bar_missing、Coolant_Flow_L_min _missing、Heat_Index _missing,这四个特征当对应特征为空值时，为 1，反之为 0。

5.1.2 数据相关性分析

Pearson 相关性分析用于度量各个变量之间线性相关程度。它的值介于-1 与 1 之间，越接近于 1 或-1 表示两个品类或单品之间强线性相关，越接近于 0 表示两个品类或单品之间越不线性相关。Pearson 相关系数的计算公式为：

$$r = \frac{\sum_{i=1}^{n}(X_i - \overline{X})(Y_i - \overline{Y})}{\sqrt{\sum_{i=1}^{n}(X_i - \overline{X})^2}\sqrt{\sum_{i=1}^{n}(Y_i - \overline{Y})^2}}$$

X、Y分别表示两个变量的观测值，X、Y分别表示X和Y的均值，n表示样本大小。 显著性 P 值，用来检验相关系数是否具有统计学上的显著性。如果 P 值小于选定的显著性水平（通常是 0.05 或 0.01），那么我们可以拒绝零假设，即认为两个变量之间存在显著的相关性。如果 P 值大于选定的显著性水平，则我们不能拒绝零假设，即认为两个变

量之间的相关性不具有显著性。

通过计算处理过后的工厂传感器模拟器数据集中24个变量之间的Person相关系数，相关系数热力图（图2）：

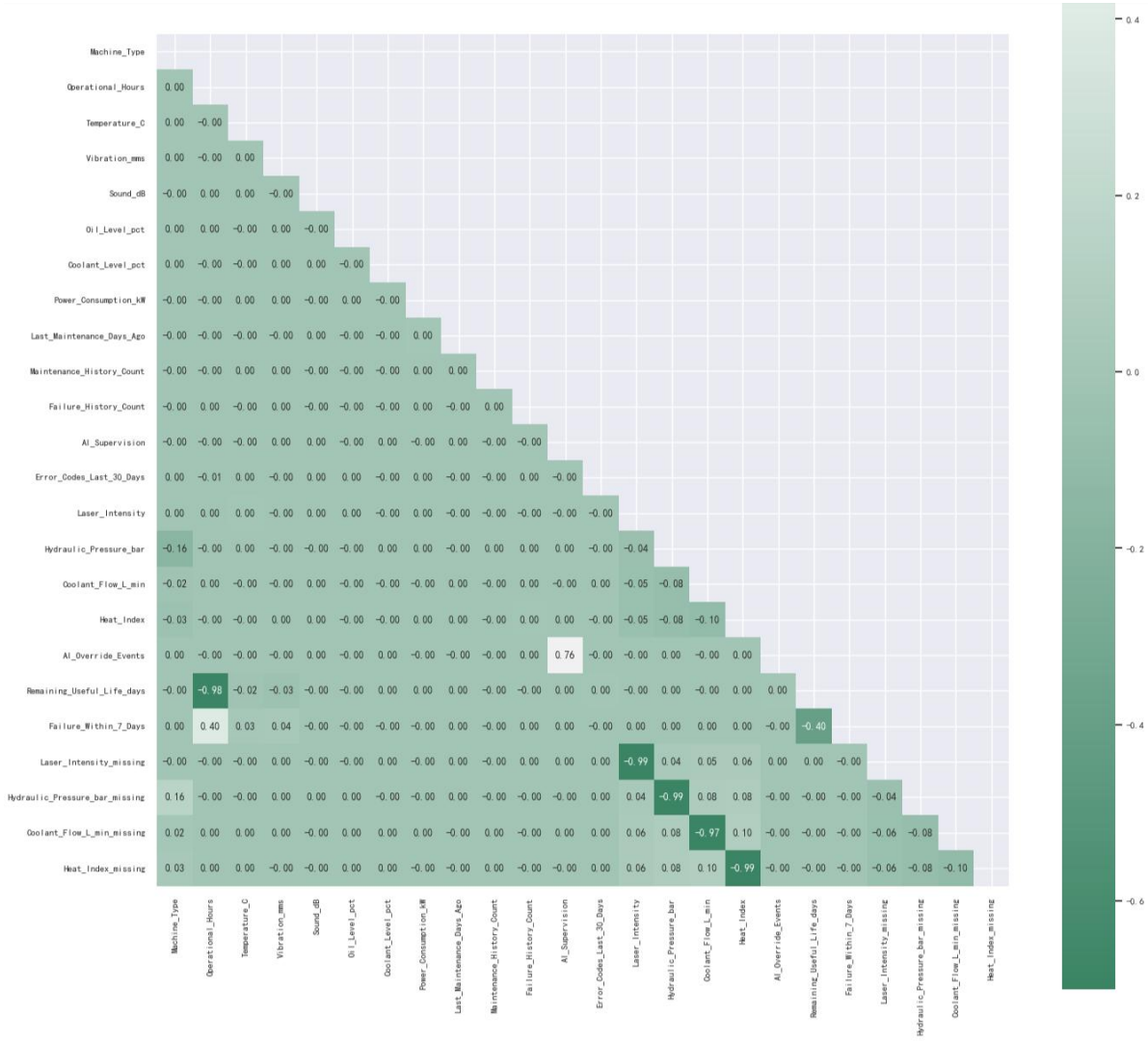

图 3 工厂传感器模拟器数据变量之间的相关系数热力图

因为Laser_Intensity_missing、Hydraulic_Pressure_bar_missing、Coolant_Flow_L_min_missing、Heat_Index_missing这四个布尔类型特征是由Laser_Intensity、Hydraulic_Pressure_bar、Coolant_Flow_L_min、Heat_Index生成，所以具有强负相关关系。

对于Remaining_Useful_Life_days、Operational_Hours具有强负相关关系，非常符合生产实际。操作时间越大，机器的剩余寿命就越少，所以，在问题A中进行分析时，去掉Remaining_Useful_Life_days这个特征。

AI_Supervision与AI_Override_Events呈现明显正相关关系。

5.1.3 GridSearchCV（网格搜索交叉验证）

GridSearchCV（网格搜索交叉验证）是一种用于系统地遍历多种参数组合，通过交叉验证确定最佳参数的方法。它的基本思想是通过将参数网格化，将所有可能的参数组合都尝试一遍，然后通过交叉验证来评估每种参数组合的性能，最终选出最优的参数组合。

5.1.4 随机森林分类模型

随机森林属于集成学习，其核心思想就是集成多个弱分类器。随机森林采用 Bagging 的思想，所谓的 Bagging 就是：

（1）每次有放回地从训练集中取出 n 个训练样本，组成新的训练集；

（2）利用新的训练集，训练得到 M 个子模型；

（3）对于分类问题，采用投票的方法，得票最多子模型的分类类别为最终的类别。

随机森林以决策树为基本单元，通过集成大量的决策树，就构成了随机森林。

通过使用 GridSearchCV 网络交叉验证法，将 roc_auc 设置为得分参数，将随机森林分类模型超参数网络设置如表 2 所示：

表 2　随机森林分类模型超参数网络

超参数	参数设置
n_estimators （最大弱学习器的个数）	100 ，150 ，200
max_features （考虑的最大特征数）	8, 12, 16, 20, 24
min_samples_split （内部节点再划分所需最小样本数）	2 ，4，6

通过五折交叉验证，得到最佳模型参数，如表 3：

表 3　最优随机森林分类模型超参数网络设置

超参数	参数设置
n_estimators （最大弱学习器的个数）	150
max_features （考虑的最大特征数）	8
min_samples_split （内部节点再划分所需最小样本数）	4

5.1.5 模型评估

通过测试集评估模型的准确性。使用统计指标（准确率、召回率、F1 值）来评价模型预测的准确性。

（1）准确率(Accuracy)：预测正确的样本数量占总量的百分比。具体的公式如下：

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}$$

（2）召回率(Recall)：在实际为正样本中，被预测为正样本所占的百分比。具体公式如下：

$$\text{Recall} = \frac{TP}{TP + FN}$$

（3）精准率(Precision)：在模型预测为正样本的结果中，真正是正样本所占的百分比。具体公式如下：

$$\text{Precision} = \frac{TP}{TP + FP}$$

（4）F1 Score：精确率（Precision）和召回率（Recall）之间的平衡点。具体公式如下：

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

上述公式中的 TP、FN、FP、TN 分别是真正类数量、假反类数量、假正类数量、真反类数量。

将随机森林模型训练后在测试集上得到的准确率、召回率、F1 作为性能指标。分别为 0.9998、0.9983、0.9991。在训练集上进行测试，得到的混淆矩阵如图 3 所示。

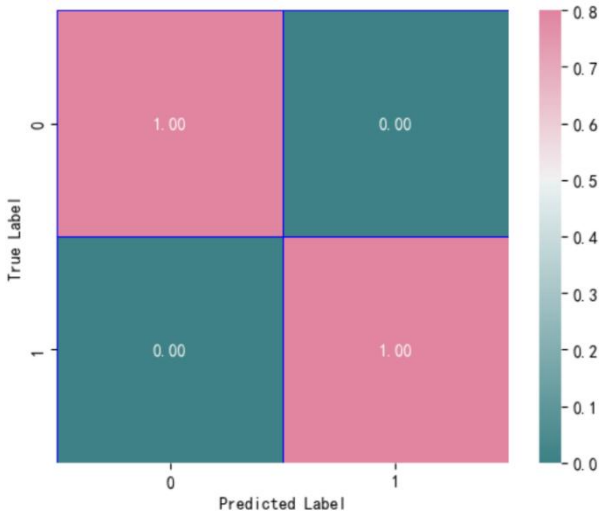

图 4 随机森林在测试集上测试效果

5.2 问题 B 的建立与求解

5.2.1 数据预处理

(1)特征分类

首先根据特征的特点将特征分为数值型、枚举型、布尔型，与问题 A 处理方式一致。

（2）数据分布与数据处理

对 15 个数值类型的变量采用归一化处理方式 对布尔类型数据采用 0-1 化处理方式。对枚举类型采用编码的方式处理。

将数据集的 80%作为训练集，20%作为测试集。

使用 KDE 方法查看训练集、测试集数据分布情况，如图 5 所示：

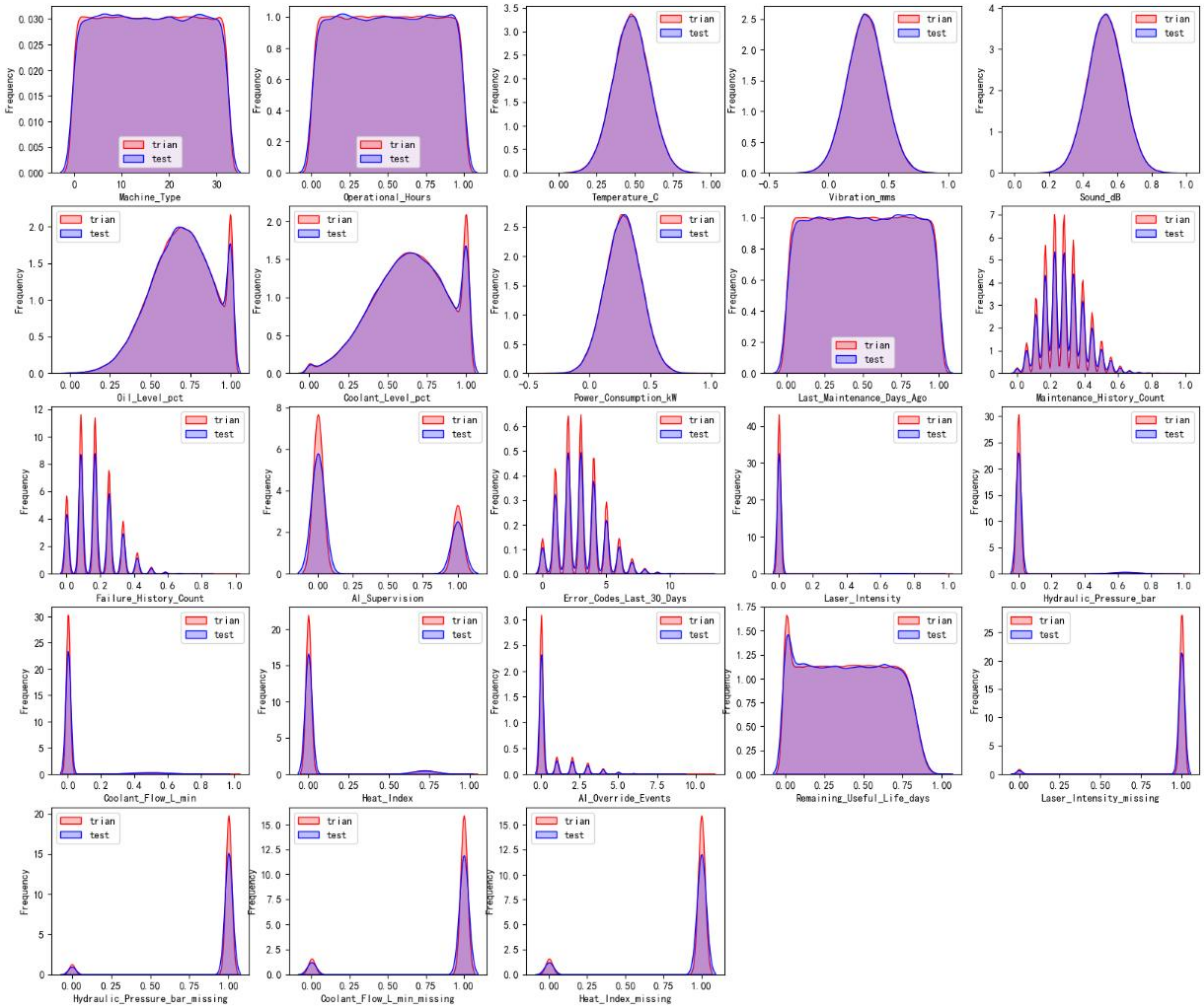

图 5 数据分布 KDE 图

（3）特征工程

特征工程操作与问题 A 一致。

5.2.2 数据相关性分析

与问题 A 保持一致。

选取'Machine_Type'，'Operational_Hours', 'Temperature_C', 'Vibration_mms', 'Sound_dB','Oil_Level_pct','Coolant_Level_pct','Power_Consumption_kW','Last_Maintenance_Days_Ago','Maintenance_History_Count','Failure_History_Count','AI_Supervision','Error_Codes_Last_30_Days','Laser_Intensity','Hydraulic_Pressure_bar','Coolant_Flow_L_min','Heat_Index','AI_Override_Events','Laser_Intensity_missing','Hydraulic_Pressure_bar_missing','Coolant_Flow_L_min_missing','Heat_Index_missing'这 22 个特征作为输入变量的特征。

5.2.2 定义数据加载器

Pytorch 框架，为了加速运算，使用的数据类型为张量。故先把数据转化为张量形式。数据为两个部分，第一个部分是枚举类型中的'Machine_Type'，第二个部分为其他变量，包括数值类型数据和布尔类型数据。之所以要分为两个部分，是因为，在模型训练前，需要将'Machine_Type'数据先嵌入为 5 维向量。以 64 个数据为一个批次，构建 DataLoader。

5.2.3 残差神经网络结构

增加神经网络的层数可以提高训练精度，但是如果只是单纯的增加网络的深度，可能会出现“梯度弥散”和“梯度爆炸”等问题。传统的解决方法则是权重的初始化和批标准化，虽然解决了梯度问题，但是深度加深了，却带来了另外的问题，就是网络性能的退化现象，可以简单的理解为，随着训练轮数（epoch）的增加，精度到达一定程度后，就开始下降了。残差连接层可以很好的解决这个问题。

所搭建的残差网络如下图所示：

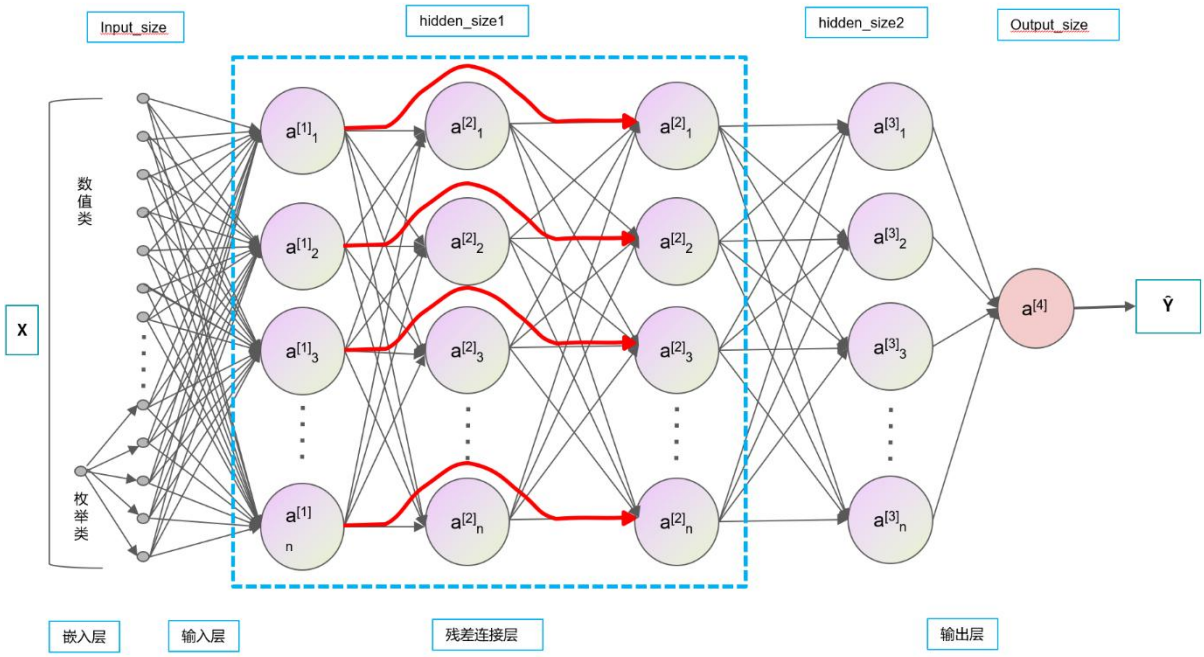

图 6 神经网络模型结构

输入维度为 26 维：由机器类型嵌入为 5 维，数值型数据 14 维，枚举数据 2 维，布尔类型数据 5 维。

输出维度 1 维：模型预测机床设备剩余使用寿命天数。

5.2.4 利用遗传算法搜寻最佳网络层数

遗传算法（Genetic Algorithm，简称 GA）起源于对生物系统所进行的计算机模拟研究，是一种随机全局搜索优化方法，它模拟了自然选择和遗传中发生的复制、交叉(crossover)和变异(mutation)等现象，从任一初始种群（Population）出发，通过随机选择、交叉和变异操作，产生一群更适合环境的个体，使群体进化到搜索空间中越来越好的区域，这样一代一代不断繁衍进化，最后收敛到一群最适应环境的个体（Individual），从而求得问题的优质解。

表 4 神经网络隐藏层层数

超参数	参数范围
Hidden_size1 （隐藏层 1）	[8,128]
Hidden_size2 (隐藏层 2)	[8,128]

表 5 最佳神经网络隐藏层层数

超参数	参数范围
Hidden_size1 （隐藏层 1）	64
Hidden_size2 (隐藏层 2)	64

5.2.5 训练神经网络

确定下最优网络层数后，开始使用最佳网络层数训练神经网络。损失函数为 MSE，采用早停法，当验证集的损失不再增加，且持续 100 轮，后，停止继续迭代。

训练的损失函数如下图：

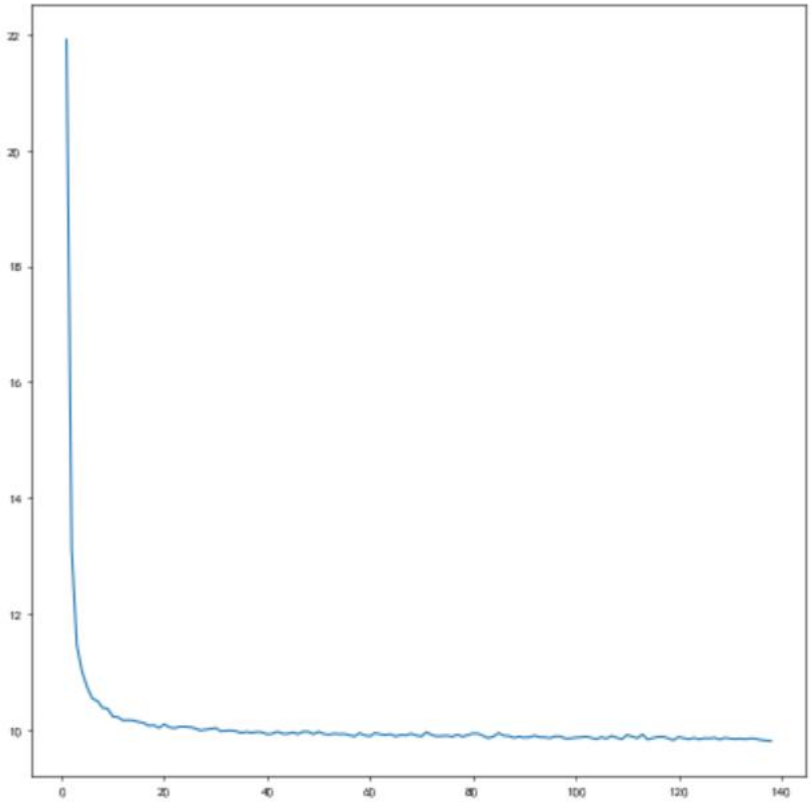

图 7 神经网络损失函数

看得出模型收敛的非常快，且稳定在较低的位点，说明拟合较好。

5.2.6 模型评估

通过测试集评估模型的准确性。使用统计指标（MSE、MAE、$RMSE$、$R2$）来评价模型预测的准确性。

（1）均方误差 (MSE)：预测值与实际值之差平方的期望值。取值越小，模型准确

度越高。公式如下：

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y})^2$$

（2）拟合优度（R^2）：将预测值跟只使用均值的情况下相比，取值范围为 0-1，R^2 取值越接近 1 说明回归拟合效果好，模型准确度越高。公式如下：

$$R^2 = \frac{SSR}{SST} = \frac{\sum_{i=1}^{n}(y_i - \overline{y})^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2}$$

其中，SSR为y的变差平方和，SST为y的总变差平方和。

使用神经网络模型预测预测机床设备的剩余使用寿命。R^2=0.97。模型预测精度检验图如图所示：

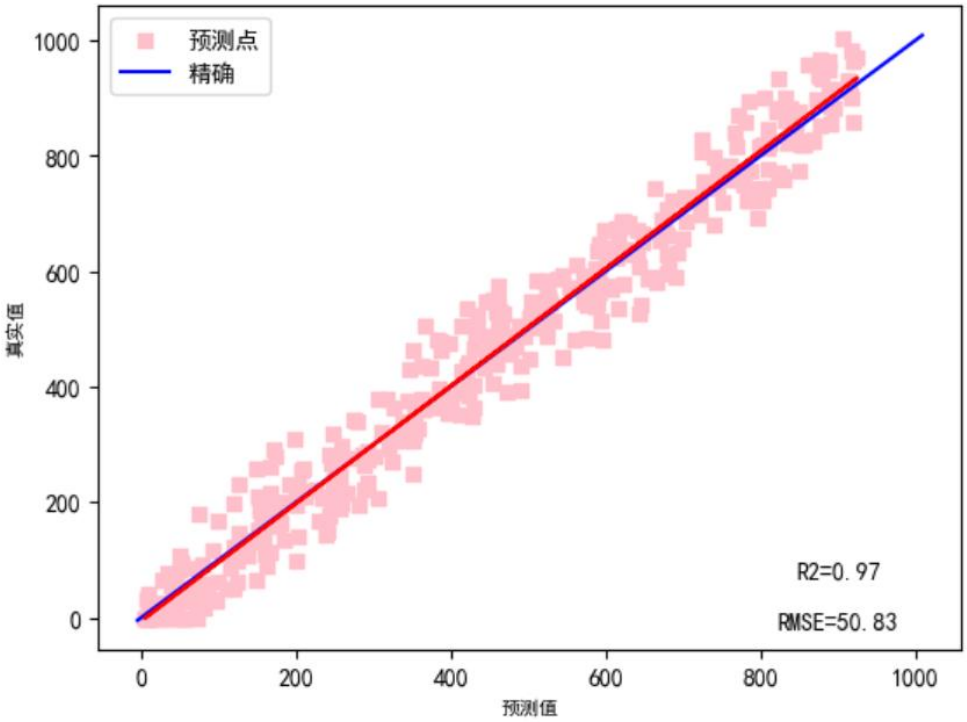

图 8 神经网络模型预测精度检验图

六、模型分析

6.1 问题 A 的模型分析

6.1.1 随机森林导出的特征重要度

在随机森林中，特征重要性通常基于两个原则进行计算：基尼不纯度(Gini Impurity)或者信息熵(Entropy)的下降量以及每个特征在所有决策树中的节点上出现的频率。具体来说，计算特征重要性的常用方法包括：

基于基尼不纯度的特征重要性: 计算每个特征在每棵决策树中节点分裂时基尼不纯度的减少量，然后取平均值或加总作为该特征的重要性得分。

基于信息熵的特征重要性: 类似基尼不纯度，计算每个特征对信息熵的减少量，然后汇总计算整体的特征重要性。

基于工业传感器数据集的 23 个特征，针对七日内机床是否损坏问题。由随机森林导出的特征重要度如图 4 所示。可见运行小时数、振动、温度、噪音与功率是随机森林模型判断机床是否会发生故障的最重要的五个特征。

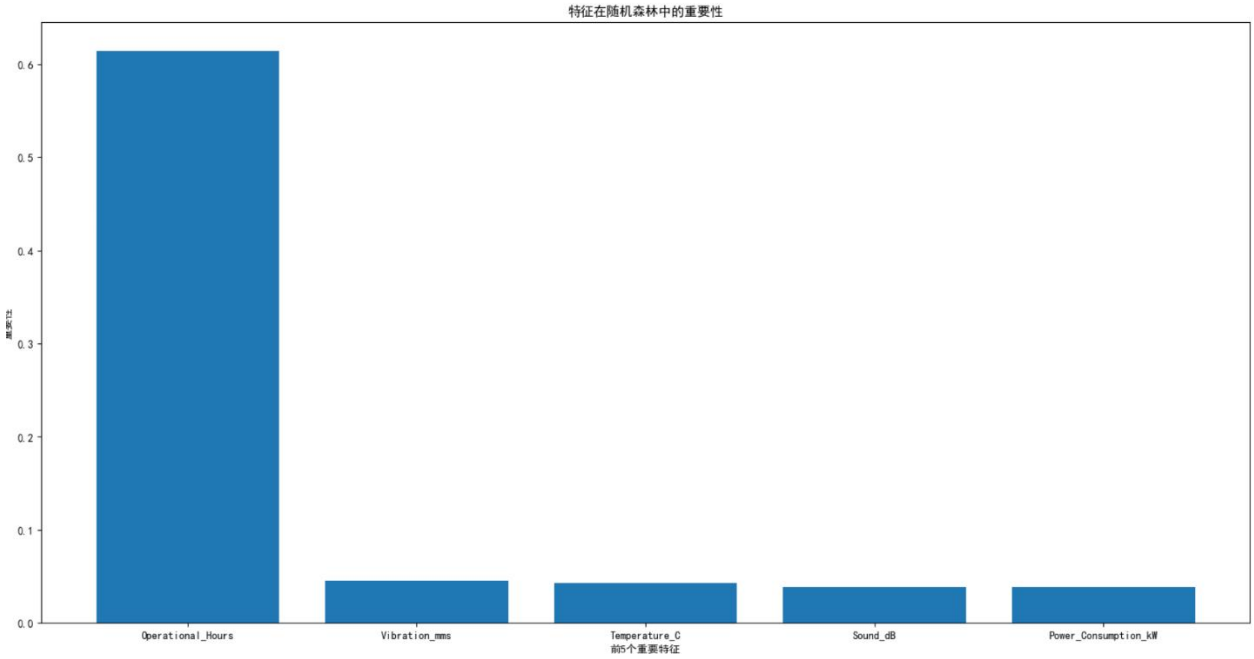

图 9 随机森林导出的特征重要度

6.1.2 基于随机森林的 SHAP 值分析

对于集成树模型来说，当做分类任务时，模型输出的是一个概率值。因此 SHAP 实

际是将输出值归因到每一个特征的 shapely 值上，换句话说，就是计算每一个特征的 shapley 值，依此来衡量特征对最终输出值的影响。用公式表示：

$$g(z^{'}) = \phi_0 + \sum_{j=1}^{M} \phi_j$$

其中 g 是解释模型，M 是输入特征的数目，ϕ是每个特征的归因值（Shapley 值），ϕ_0是一个常数。公式中的难点是求ϕ_j。对某个特征 j，需要针对所有可能的特征组合（包括不同顺序）计算 shapley 值，然后加权求和，即：

$$\phi_j(val) = \sum_{S \subseteq \{x_1,\ldots,x_p\}\{x_j\}} \frac{|S|!\,(p-|S|-1)!}{p!}\left(val\left(S \cup \{x_j\}\right) - val(S)\right)$$

其中 S 是模型中使用的特征的子集，x 是要解释的样本的特征值的向量，p 为特征数量 ， val(S)指在特征组合 S 下的模型输出值，解释一下权重，一共 p 个特征，则在考虑顺序的情况下这 p 个特征共有 p!种组合，固定了某个特征 j，则剩余的有(p-| S |-1)!S!种组合。

本节利用 5.1 节中所构建的随机森林模型，在随机抽取训练集中 4000 条数据作为背景数据样本，在测试集中抽取 500 条数据作为解释样本。绘制 summary 图。如图 5 所示：

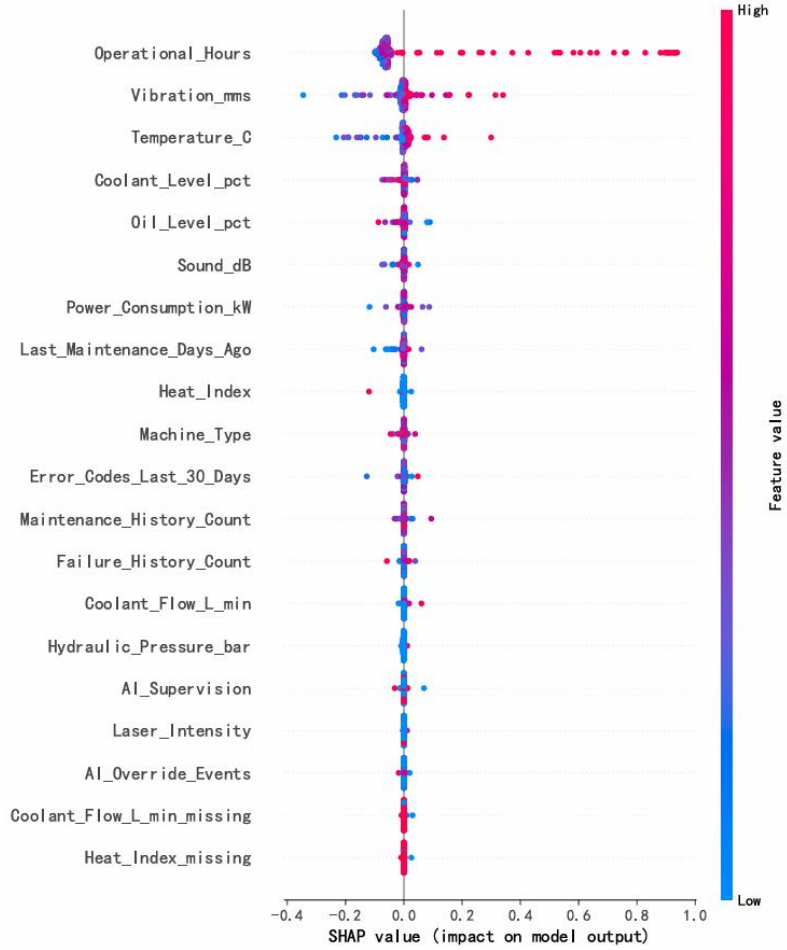

图 10 随机森林模型 SHAP 分析 Summary 图

纵轴按照所有样本的 SHAP 值之和对特征排序，横轴是 SHAP 值，即特征对模型输出的影响分布；每个点代表一个样本，样本量在纵向堆积，颜色表示特征值，红色对应高值，蓝色对应低值。

可以看到当 Operation_Hours 越大时，其对应的 SHAP 值越大，即对模型的正影响作用越大。反应在实际生产中就是，机器使用的时间越久，出现故障的概率越大。Vibration_mms 与 Tempature_C 越大时，其对应的 SHAP 值越大，即对模型的正影响越大。反映在实际生产过程中，可能是因为故障可能会引起机械异常升温、异常振动等情况。

SHAP 值还能对单个样本进行分析。如图 6、7 所示：

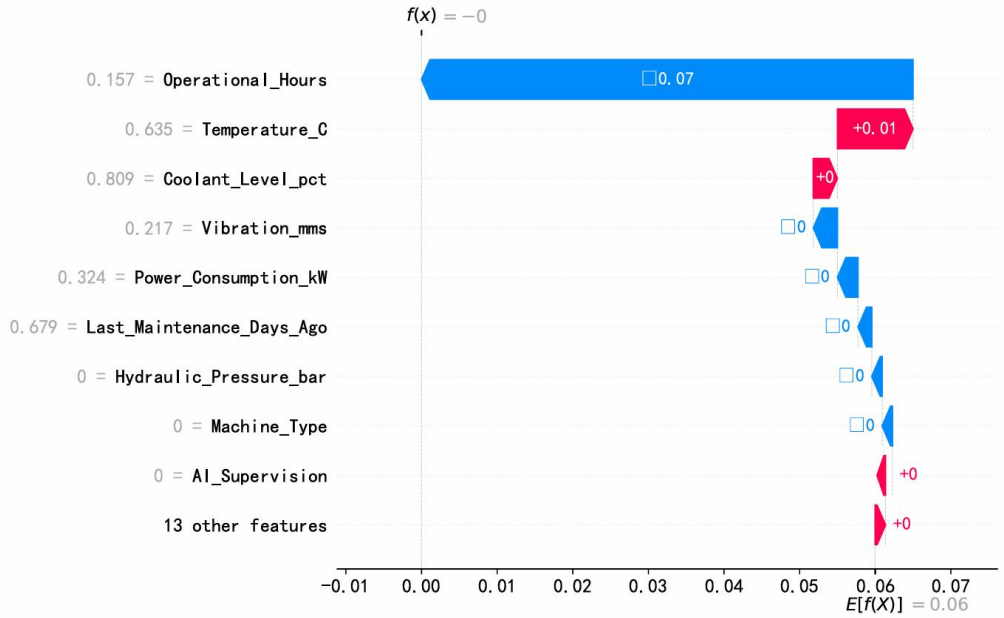

图 11 某一无故障设备的 SHAP 值分析

可看到基础值为 0.06，意味着，原数据集中，故障的期望值为 0.06，其中样本的操作时间值为 0.157（经过处理后，实际为 178 天），属于操作时间较短的，故模型认为其不会出现故障，在原有的基础值上扣去 0.07，但温度较高，对模型有微弱正向作用为 0.01，但正向作用尚不能“抵消”负向作用，故在综合下，模型认为这个机床设备七日之内不会发生故障。

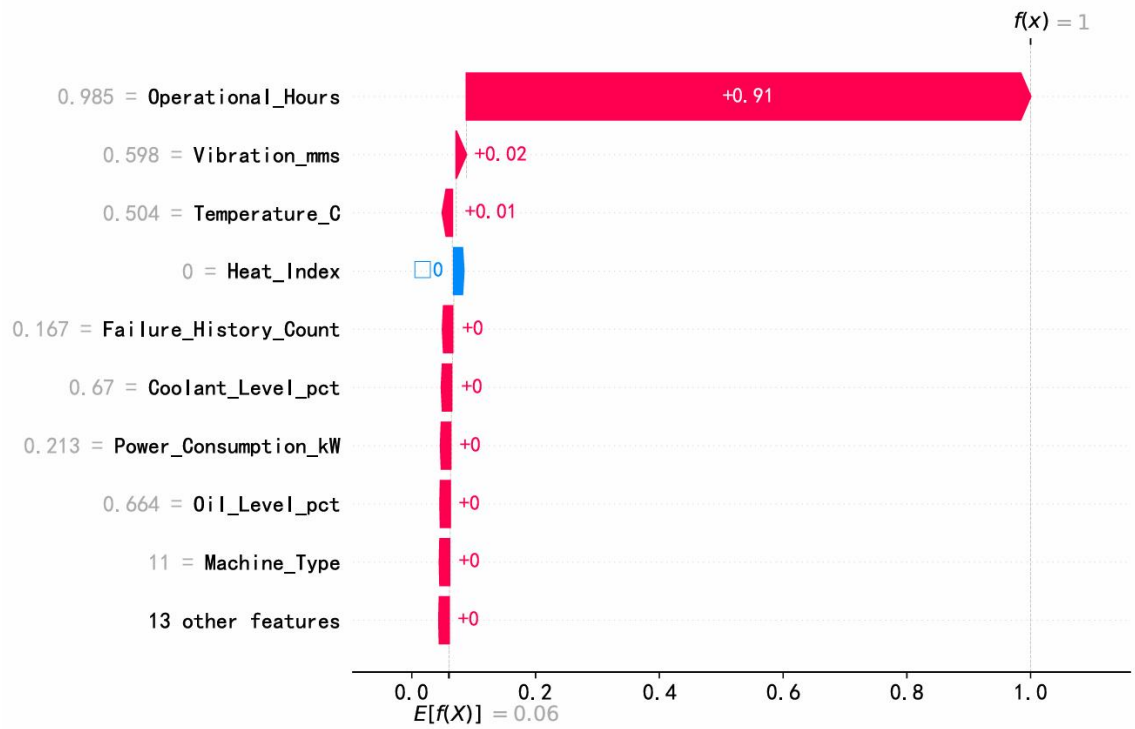

图 12 某一故障设备的 SHAP 值分析

可看到基础值为 0.06，意味着，原数据集中，故障的期望值为 0.06，，其中样本的操作时间值为 0.985（经过处理后，实际为 1116 天），属于操作时间很长的，故模型认为会出现故障，在原有的基础值上加上 0.91，且温度较高，对模型有微弱正向作用为 0.01，其他特征对预测影响不大。

6.2 问题 B 的模型分析

本节基于神经网络模型，使用 SHAP 方法，分析各个特征对机床设备剩余使用寿命的影响。在训练集中选择 500 条样本作为背景数据，训练集中的 200 条数据作为解释数据，构建 SHAP 分析。

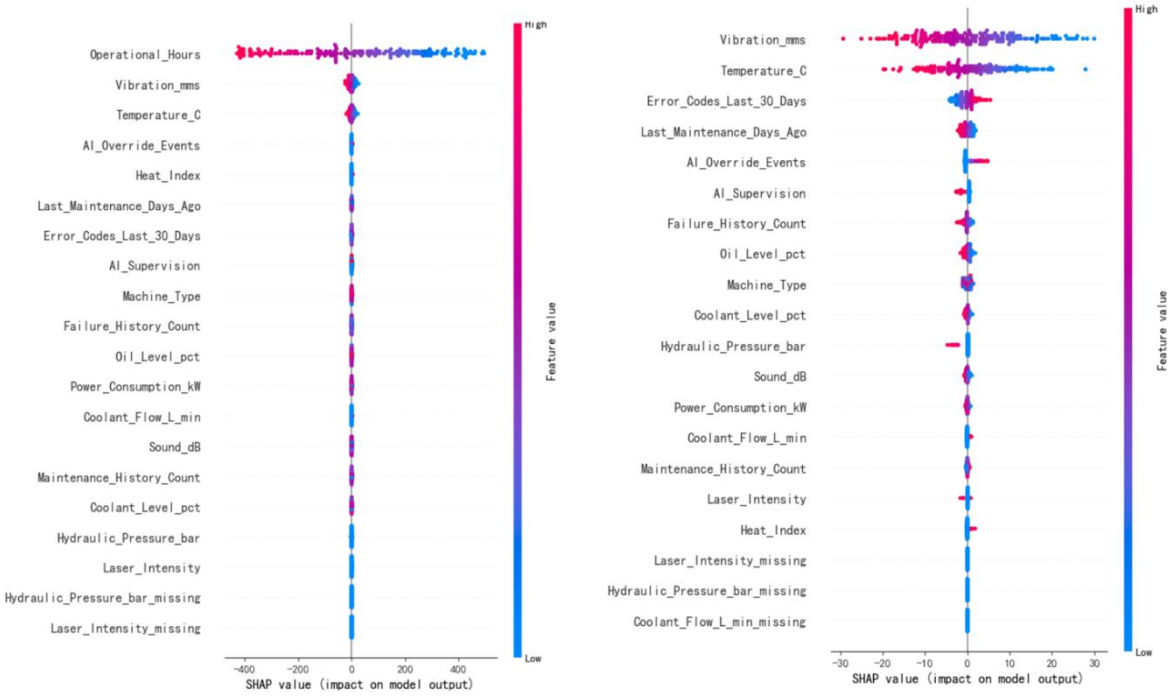

图 13 神经网络模型 SHAP 分析 Summary 图

从图 13 左图中，可观察到，操作时间是对剩余使用寿命影响最大的因素。当点的颜色越红，说明操作时间越久，SHAP 值为负值，且绝对值越大，说明对剩余使用寿命的负影响作用越大。在实际生产环境中，使用越久的设备，其剩余的使用寿命越短。

为了进一步分析其他因素对机床设备使用寿命的影响，将 Operational_Hours 特征去除后，重新进行分析。如右图所示，可以看到 Temperature_C （温度）, Vibration_mms（振动）对预测影响较大，振动幅度越大、温度越高，机器的剩余使用寿命越小。

此外，还注意到 Error_Codes_Last_30_Days（最近 30 天故障编码）对模型同样有影响，且值越小，机床设备的剩余使用寿命越短。可能说明，错误编码是故障的严重程度，

等级数字越小，说明故障越严重。

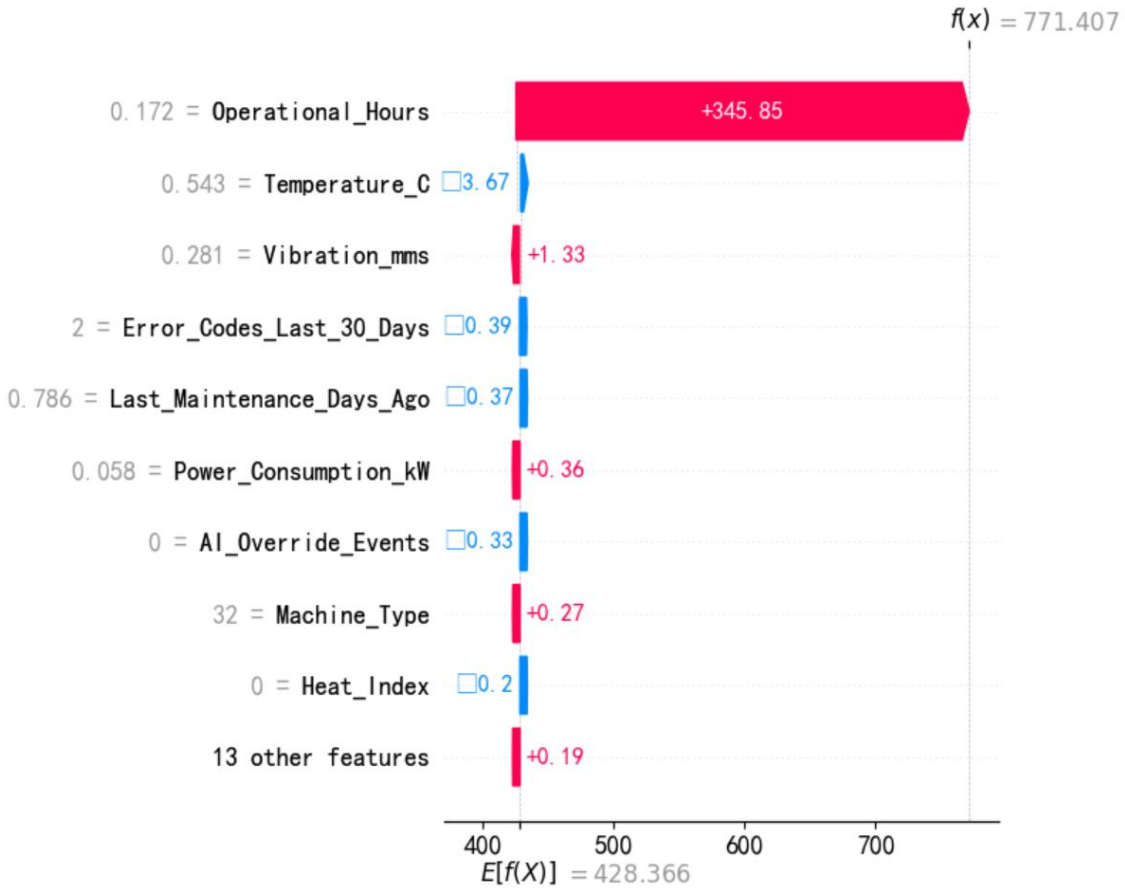

图 14 神经网络模型对单一样本的 SHAP 分析

从图 14 中可以得知，该机床设备使用时间较短，因此剩余使用寿命较长，但此时温度较高，可能会缩短该机床设备的使用寿命，近 30 天有出现一个编号为 2 的故障，较为严重，可能会对机床设备使用寿命有所影响。

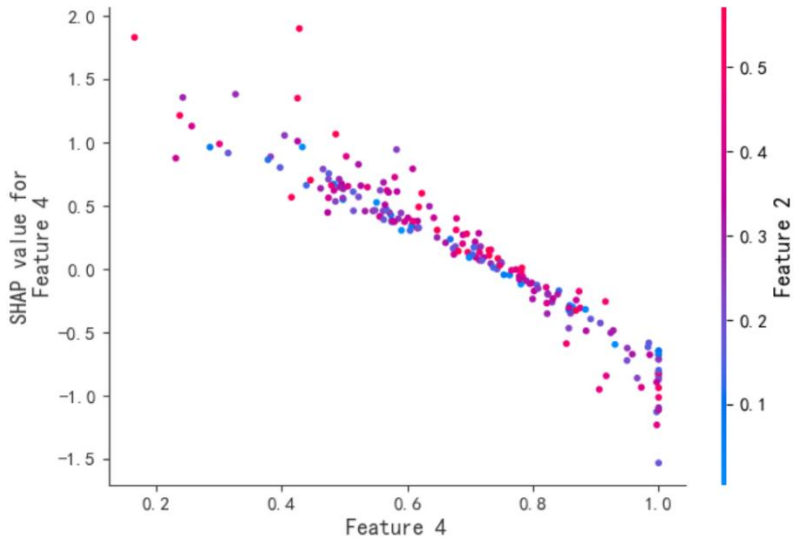

图 15 神经网络模型的振动、油位特征依赖分析

图 15 是神经网络对振动、油位的特征以来分析。可以看到，当油位位于较低点时，振动时比较大的。

七、模型的评价

6.1 模型优点

（1）本文充分考虑数据质量，利用了绝大多数特征，从而提高了模型运行效率和准确率。

（2）本文采用了 GridSearchCV 网络交叉验证的方法以及遗传算法，经过参数选择和交叉验证得到了最佳的模型参数，提高模型性能。

6.2 模型缺点

（1）机床设备作为一个复杂系统，其问题复杂度较高，神经网络模型有可能陷入局部最优解。

（2）SHAP 分析是基于博弈论的方法，其虽具有参考价值，但并不能代替实际生产分析判断。

参考文献

[1] Lundberg, Scott M., and Su-In Lee. "A unified approach to interpreting model predictions." Advances in neural information processing systems 30 (2017).

[2] Interpreting Machine Learning Models With SHAP——A Guide With Python Examples And Theory On Shapley Values https://leanpub.com/shap

[3] 耿明,张海沧,康丽齐,等. 基于主成分分析和随机森林回归模型的工艺装备轴承剩余寿命预测 [J]. 城市轨道交通研究,2023,26(4):12-16.

[4] 吴超.皮尔逊相关系数在台区线损异常排查中的应用研究[J].光源与照明,2022(8):141-143.

[5] 王永.工业4.0时代打造智慧工厂技术的应用与研究[C]//天津市电子学会.第三十八届中国（天津）2024’IT、网络、信息技术、电子、仪器仪表创新学术会议论文集.德中(天津)技术发展股份有限公司.

附录

问题 A 部分代码

```
def view_data_distribution(dataframe,nrows=4,ncols=4,path=False):
    ticks = list(dataframe.columns)[:]

    plt.figure()
    fig,axes = plt.subplots(nrows=nrows,ncols=ncols,figsize=(ncols*5,nrows*5))
    axes = axes.flatten()
    for i,tick in enumerate(ticks):
        ax = axes[i]
        ax.boxplot(dataframe[tick],
                   patch_artist=True,
                   flierprops=dict(marker='.', markersize=1),
                   widths=0.5,   #箱子宽度
                   boxprops={"facecolor": "skyblue", "edgecolor": "white",
                             "linewidth": 0.5}
                  )
        ax.set_xlabel(tick)
        ax.set_xticks([1],[])   # 设置 x 轴刻度位置
    if path:
        plt.savefig(path,dpi=600)#保存图片，分辨率为 600
    plt.show()

# 查看数据相关性
def
view_corr_plot(dataframe,label='Failure_Within_7_Days',path1=False,path2=False,ifpairplot
=False,figsize=(8,8)):
    corr = dataframe.corr()
    rc = {'font.sans-serif': 'SimHei',
          'axes.unicode_minus': False}
    plt.figure(figsize=figsize)
```

```
    sns.set(font_scale=0.7,rc=rc)   # 设置字体大小

    mask = np.zeros_like(corr,dtype=bool)     #生成与相关系数矩阵同形的全 False 布尔矩阵。
    mask[np.triu_indices_from(mask)] = True
    cmap = sns.diverging_palette(150,10,as_cmap=True)     #设置颜色映射，创建蓝-白-红的发散调色板，突出正负相关系数的对比，中间值接近白色。

    sns.heatmap(corr,
                mask=mask,
                annot=True,   # 显示相关系数的数据
                center=0.5,   # 居中
                fmt='.2f',   # 只显示两位小数
                vmin=-0.7, vmax=0.5,   # 设置数值最小值和最大值
                xticklabels=True, yticklabels=True,   # 显示 x 轴和 y 轴
                square=True,   # 每个方格都是正方形
                cbar=True,   # 绘制颜色条
                cmap=cmap   # 设置热力图颜色
                )
    if path1:
        plt.savefig(path1,dpi=600)#保存图片，分辨率为 600
    plt.ion() #显示图片

    if ifpairplot:
        corr_label = corr[label].sort_values(key=abs,ascending=False)
        corr_label_list = list(corr_label.index[1:4])
        sns.pairplot(corr[[corr_label_list[0],corr_label_list[1],corr_label_list[2],label]])
        if path2:
            plt.savefig(path2,dpi=600)
        plt.show()

def KDE_plot(train_data,test_data,dist_cols):
```

```
    '''
    train_data 与 test_data 均为 dataframe 类型
    '''
    dist_cols = dist_cols
    dist_rows = len(test_features.columns)
    plt.figure(figsize=(4*dist_cols,4*(dist_rows//dist_cols)+1))
    i=1
    for col in test_features.columns:
        ax = plt.subplot(dist_cols,(dist_rows//dist_cols)+1,i)
        ax = sns.kdeplot(train_data[col],color='r',fill=True)
        ax = sns.kdeplot(test_features[col],color='b',fill=True)
        ax.set_xlabel(col)
        ax.set_ylabel('Frequency')
        ax = ax.legend(['trian','test'])
        i+=1
    plt.show()

# 绘制混淆矩阵
def Confusion_Matrix(conf_matrix):
    conf_matrix_f = conf_matrix.copy().astype(float)
    row_num = conf_matrix_f.shape[0]
    for row in range(row_num):
        conf_matrix_f[row,:] = conf_matrix_f[row,:]/np.sum(conf_matrix_f[row,:])
    cmap = sns.diverging_palette(200,0,as_cmap=True)    #设置颜色映射，创建蓝-白-红的发散调色板，突出正负相关系数的对比，中间值接近白色。
    sns.heatmap(conf_matrix_f,
                annot=True,
                center=0.5,
                fmt='.2f',
                linewidth=0.5,  # 设置每个单元格的距离
                linecolor='blue',  # 设置间距线的颜色
                vmin=0, vmax=0.8,  ## 设置数值最小值和最大值的饱和值
                xticklabels=True, yticklabels=True,  # 显示 x 轴和 y 轴
```

```
                    square=True,  # 每个方格都是正方形
                    cbar=True,  # 绘制颜色条
                    cmap=cmap,  # 设置热力图颜色
                    )
    # 添加轴标题（可选）
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")

# 根据第一个列表的排序顺序来同步调整第二个列表的顺序
def sort_lists(list1, list2):
    # 将两个列表的对应元素组合成元组
    zipped = zip(list1, list2)
    # 根据 list1 的值对元组进行排序
    sorted_zipped = sorted(zipped, key=lambda x: x[0],reverse=True)
    # 解压排序后的元组到两个列表
    sorted_list1, sorted_list2 = zip(*sorted_zipped)
    # 将结果转回列表类型
    return list(sorted_list1), list(sorted_list2)

# 归一化函数
def scaled_func(col):
    num_max = abs(col).max()
    num_min = 0 if col.min()<=0 else col.min()
    return (col - num_min)/(num_max - num_min)

# 归一化函数数据
def scaled_func_info(col):
    num_max = abs(col).max()
    num_min = 0 if col.min()<=0 else col.min()
    return num_max,num_min

# 使用归一化函数(A)
```

```
def scaled_func_used(df):
    df_copy = df.copy()
    num_dict = {'Operational_Hours':[100000,0],
                'Temperature_C':[126.17,0],
                'Vibration_mms':[32.47,0],
                'Sound_dB':[120.7,23.65],
                'Oil_Level_pct':[100,0],
                'Coolant_Level_pct':[100,0],
                'Power_Consumption_kW':[544.2,0],
                'Last_Maintenance_Days_Ago':[365,0],
                'Maintenance_History_Count':[18,0],
                'Failure_History_Count':[12,0],
                'Remaining_Useful_Life_days':[1133,0],
                'Laser_Intensity':[114.32,0],
                'Hydraulic_Pressure_bar':[185.38,0],
                'Coolant_Flow_L_min':[80.79,0],
                'Heat_Index':[693.55,0]}

    for col in num_dict:
        df_copy[col] = df_copy[col].astype(float)
        df_copy.loc[:,col] = (df.loc[:,col] - num_dict[col][1])/(num_dict[col][0] -
num_dict[col][1])
    return df_copy

# 使用归一化函数(B)
def scaled_func_used_B(df):
    df_copy = df.copy()
    num_dict = {'Operational_Hours':[100000,0],
                'Temperature_C':[126.17,0],
                'Vibration_mms':[32.47,0],
                'Sound_dB':[120.7,23.65],
                'Oil_Level_pct':[100,0],
                'Coolant_Level_pct':[100,0],
```

```
                    'Power_Consumption_kW':[544.2,0],
                    'Last_Maintenance_Days_Ago':[365,0],
                    'Maintenance_History_Count':[18,0],
                    'Failure_History_Count':[12,0],
                    'Laser_Intensity':[114.32,0],
                    'Hydraulic_Pressure_bar':[185.38,0],
                    'Coolant_Flow_L_min':[80.79,0],
                    'Heat_Index':[693.55,0]}

    for col in num_dict:
        df_copy[col] = df_copy[col].astype(float)
        df_copy.loc[:,col] = (df.loc[:,col] - num_dict[col][1])/(num_dict[col][0] -
num_dict[col][1])
    return df_copy

# 使用编码器
def safe_transform_with_mapping(le, data):
    """
    使用映射字典处理未知类别
    le: 训练好的 LabelEncoder
    data: 要转换的数据(Series)
    """
    # 创建类别到编码的映射字典
    category_mapping = dict(zip(le.classes_, le.transform(le.classes_)))

    # 添加未知类别处理（分配新编码）
    unknown_code = len(le.classes_)

    # 使用 map 函数应用转换
    return data.map(lambda x: category_mapping.get(x, unknown_code))

# 创建在图中添加文本注释的方法 text
def text(ax,x, y, text):
```

```
    # 在指定的 (x, y) 位置添加文本注释
    ax.text(
        x,
        y,
        text,  # 要显示的文本内容
        zorder=100,  # 绘制顺序，较大的值会在其他元素之上
        ha='center',  # 水平对齐方式，居中对齐
        va='top',  # 垂直对齐方式，顶部对齐
        weight='bold',  # 字体加粗
        color='black',  # 文本颜色为黑色
        style='italic',  # 文字样式为斜体
    )
    def A_model_use(dataframe):
        X_cols_name = ['Machine_Type',
                       'Operational_Hours',
                       'Temperature_C',
                       'Vibration_mms',
                       'Sound_dB',
                       'Oil_Level_pct',
                       'Coolant_Level_pct',
                       'Power_Consumption_kW',
                       'Last_Maintenance_Days_Ago',
                       'Maintenance_History_Count',
                       'Failure_History_Count',
                       'AI_Supervision',
                       'Error_Codes_Last_30_Days',
                       'Laser_Intensity',
                       'Hydraulic_Pressure_bar',
                       'Coolant_Flow_L_min',
                       'Heat_Index',
                       'AI_Override_Events',
                       'Remaining_Useful_Life_days']
        df = dataframe.loc[:,X_cols_name]
```

```
# 转化 bool 类型数据
AI_Supervision_num = []
for i in df['AI_Supervision']:
    if i == True:
        AI_Supervision_num.append(1)
    else:
        AI_Supervision_num.append(0)
df['AI_Supervision'] = np.array(AI_Supervision_num)
# 给机器类型编号
machine_type = []
for i in df['Machine_Type']:
    if i not in machine_type:
        machine_type.append(i)
machine_type_num = [i for i in range(len(machine_type))]
machine_type_dict = dict(zip(machine_type,machine_type_num))
df['Machine_Type'] = df['Machine_Type'].map(machine_type_dict)
# 添加
'Laser_Intensity','Hydraulic_Pressure_bar','Coolant_Flow_L_min','Heat_Index'的缺失指示器
for col in
['Laser_Intensity','Hydraulic_Pressure_bar','Coolant_Flow_L_min','Heat_Index']:
    df[f'{col}_missing'] = df[col].isna().astype(int)
# 用 0 填充空值
FILL_VALUE = 0
df_filled = df.fillna(FILL_VALUE)
# 处理数值类型数据
df_filled = scaled_func_used(df_filled)
feature_cols = ['Machine_Type',
                'Operational_Hours',
                'Temperature_C',
                'Vibration_mms',
                'Sound_dB',
                'Oil_Level_pct',
                'Coolant_Level_pct',
```

```
                                'Power_Consumption_kW',
                                'Last_Maintenance_Days_Ago',
                                'Maintenance_History_Count',
                                'Failure_History_Count',
                                'AI_Supervision',
                                'Error_Codes_Last_30_Days',
                                'Laser_Intensity',
                                'Hydraulic_Pressure_bar',
                                'Coolant_Flow_L_min',
                                'Heat_Index',
                                'AI_Override_Events',
                                # 'Remaining_Useful_Life_days',
                                'Laser_Intensity_missing',
                                'Hydraulic_Pressure_bar_missing',
                                'Coolant_Flow_L_min_missing',
                                'Heat_Index_missing']
        A_models = pickle.load(open("./Models/A_model2.pkl","rb"))
        y_pred = A_models.predict(df_filled.loc[:,feature_cols].values)
        return y_pred

    # 训练函数（网格搜寻）
    def train_model_classify(model,X:pd.DataFrame,y:pd.Series,param_grid:dict, splits=5,
repeats=5):

        # create cross-validation method
        rkfold = RepeatedKFold(n_splits=splits, n_repeats=repeats)

        # perform a grid search if param_grid given
        # setup grid search parameters
        gsearch = GridSearchCV(model,
                               param_grid,
                               cv=rkfold,
                               scoring="roc_auc",
```

```
                                    verbose=1,
                                    return_train_score=True)

# search the grid
gsearch.fit(X, y)

# extract best model from the grid
model = gsearch.best_estimator_
best_idx = gsearch.best_index_

# get cv-scores for best model
grid_results = pd.DataFrame(gsearch.cv_results_)
cv_mean = abs(grid_results.loc[best_idx, 'mean_test_score'])
cv_std = grid_results.loc[best_idx, 'std_test_score']

# combine mean and std cv-score in to a pandas series
cv_score = pd.Series({'mean': cv_mean, 'std': cv_std})

# predict y using the fitted model
y_pred = model.predict(X)

# print stats on model performance
print('----------------------')
print(model)
print(f"模型最佳超参数组合为：",gsearch.best_params_)
print('----------------------')
print('score=', model.score(X, y))
print('f1 值=', calculate_f1(y, y_pred))
print('召回率=', calculate_recall(y, y_pred))
print('准确率=',accuracy_score(y,y_pred))
print('cross_val: mean=', cv_mean, ', std=', cv_std)

# 生成混淆矩阵
```

```
        conf_matrix = confusion_matrix(y,y_pred)
        Confusion_Matrix(conf_matrix)



        return model, cv_score, grid_results

# 训练函数（无网格搜寻）
def train_model_claasify_nosearch(model,X:np.array,y:np.array):
        model.fit(X,y)

        # predict y using the fitted model
        y_pred = model.predict(X)

        # print stats on model performance
        print('----------------------')
        print(model)
        print('----------------------')
        print('f1 值=', calculate_f1(y, y_pred))
        print('召回率=', calculate_recall(y, y_pred))
        print('准确率=',accuracy_score(y,y_pred))

        # 生成混淆矩阵
        conf_matrix = confusion_matrix(y,y_pred)
        Confusion_Matrix(conf_matrix)

        return model

# 评分函数
# 计算召回率
def calculate_recall(y_true, y_pred):
        """
        计算宏平均召回率（支持二分类和多分类）
```

```
        参数:
        y_true -- 真实标签数组 (n_samples,)
        y_pred -- 预测标签数组 (n_samples,)

        返回:
        recall -- 宏平均召回率
        """
        # 获取所有唯一类别
        classes = set(y_true) | set(y_pred)
        recall_scores = []

        for cls in classes:
            # 计算真阳性、假阴性
            tp = sum((y_true == cls) & (y_pred == cls))
            fn = sum((y_true == cls) & (y_pred != cls))

            # 计算当前类别的召回率
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0
            recall_scores.append(recall)

        # 计算宏平均召回率
        macro_recall = sum(recall_scores) / len(recall_scores)
        return macro_recall

        return np.sqrt(sum_sq/n)

    # 计算 F1 值
    def calculate_f1(y_true, y_pred):
        """
        计算宏平均 F1 分数（支持二分类和多分类）

        参数:
```

```
    y_true -- 真实标签数组 (n_samples,)
    y_pred -- 预测标签数组 (n_samples,)

    返回:
    f1 -- 宏平均 F1 分数
    """
    # 获取所有唯一类别
    classes = set(y_true) | set(y_pred)
    f1_scores = []

    for cls in classes:
        # 计算真阳性、假阳性、假阴性
        tp = sum((y_true == cls) & (y_pred == cls))
        fp = sum((y_true != cls) & (y_pred == cls))
        fn = sum((y_true == cls) & (y_pred != cls))

        # 计算精确率和召回率
        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0

        # 计算当前类别的 F1 分数
        f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0
        f1_scores.append(f1)

    # 计算宏平均 F1
    macro_f1 = sum(f1_scores) / len(f1_scores)
    return macro_f1

def test_model_classify(model,X,y):
    y_pred = model.predict(X)

    # 计算准确度
```

```
        accuracy = accuracy_score(y,y_pred)

        # 生成混淆矩阵
        conf_matrix = confusion_matrix(y,y_pred)
        Confusion_Matrix(conf_matrix)

        return accuracy,conf_matrix

    backgroud_np = X_train[:4000]
    explain_X_np = X_test[:500]   # 选择要解释的样本

    # 创建 SHAP 解释器,test_features 背景
    A_explainer = shap.Explainer(f,backgroud_np)

    shap_values_numpy = A_explainer.shap_values(explain_X_np)
    # 计算 shap 值为 Explanation 格式
    shap_values_raw = A_explainer(explain_X_np)

    # feature_names = features.columns
    feature_names = feature_cols

    # 手动创建一个 shap.Explanation 对象，并传递特征名
    shap_values_Explanation = shap.Explanation(values=shap_values_raw.values,


base_values=shap_values_raw.base_values,

                                                    data=explain_X_np,   # 样本数据

                                                    feature_names=feature_names)   #
特征名称
    plt.figure(figsize=(20, 16), dpi=1200)
```

```
# 使用 shap_values_Explanation 绘制摘要图，显示每个特征的影响力

shap.summary_plot(shap_values_Explanation, explain_X_np,
feature_names=feature_names, plot_type="dot", show=False)

plt.savefig("./images/SHAP_Summary_Plot.pdf", format='pdf', bbox_inches='tight')

plt.show()
```

问题 B 部分代码

```
class ResidualBlock(nn.Module):
    def __init__(self, hidden_dim1,hidden_dim2):
        super().__init__()
        self.block = nn.Sequential(     #全连接块
            nn.Linear(hidden_dim1, hidden_dim2),
            nn.BatchNorm1d(hidden_dim2),
            nn.ReLU(),
            nn.Linear(hidden_dim2,hidden_dim1),
            nn.BatchNorm1d(hidden_dim1),
            nn.ReLU()
        )
        self.fc1 = nn.Linear(hidden_dim1,hidden_dim1)

    def forward(self, x):
        residual = x
        out = self.block(x)
        out = out + residual    # 跳跃连接
        out = nn.ReLU()(out)
        out = self.fc1(out)
        return out

class ResNet_regression(nn.Module):
    def __init__(self, num_dim,embed_dim,h1,h2,output_dim):
        super().__init__()
```

```
        # 枚举特征的嵌入层
        self.enum_embed = nn.Embedding(33, embed_dim) # 33 是说明有 33 个机器种类

        input_dim = num_dim + embed_dim

        self.linear_in = nn.Sequential(
            nn.Linear(input_dim, h1),
            nn.BatchNorm1d(h1),
            nn.ReLU()
        )
        self.res_blocks = nn.Sequential(
            ResidualBlock(h1,h2),
            nn.Linear(h1,h2),
            nn.ReLU(),
            nn.Linear(h2,h2),
            nn.ReLU(),
            nn.Linear(h2,h2),
            nn.ReLU()
        )
        self.linear_out = nn.Linear(h2, output_dim)

    def forward(self, x_num,x_enum):
        x_enum = self.enum_embed(x_enum)
        x = torch.cat((x_num, x_enum), dim=1)
        x = self.linear_in(x)
        x = self.res_blocks(x)
        x = self.linear_out(x)
        return x #x.squeeze(1)

#训练函数
def train_model(model,train_loader,val_loader,patience=100,
```

```
max_epochs=1000,save_path='./Models/B_model2.pth'):
        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        model.to(device)
        optimizer = optim.Adam(model.parameters(), lr=0.001)
        criterion = nn.MSELoss()   # 无 L2 约束
        best_loss = float('inf')   #表示初始化一个无穷大数
        epochs_no_improve = 0
        loss_dict = dict()
        loss_dict['train_loss_list']=[]
        loss_dict['val_loss_list']=[]

        for epoch in range(max_epochs):
            model.train()
            train_loss = 0.0
            for X_num_batch,X_enum_batch, y_batch in train_loader:
                X_num_batch,X_enum_batch, y_batch =
X_num_batch.to(device),X_enum_batch.to(device), y_batch.to(device)
                optimizer.zero_grad()
                outputs = model(X_num_batch,X_enum_batch)
                loss = criterion(outputs, y_batch) #无 L2 约束
                # loss = bounded_loss(outputs,y_batch,0,100,5) #有 L2 约束
                loss.backward()
                optimizer.step()
                train_loss += loss.item()
            # 验证集评估
            model.eval()
            val_loss = 0.0
            with torch.no_grad():
                for X_num_val_batch,X_enum_val_batch, y_val_batch in val_loader:
                    X_num_val_batch,X_enum_val_batch, y_val_batch =
X_num_val_batch.to(device),X_enum_val_batch.to(device), y_val_batch.to(device)
                    outputs = model(X_num_val_batch,X_enum_val_batch)
                    val_loss += criterion(outputs, y_val_batch).item()
```

```
                # val_loss += bounded_loss(outputs, y_val_batch,0,100,5).item() #
有 L2 约束

            avg_train_loss = train_loss / len(train_loader)
            avg_val_loss = val_loss / len(val_loader)
            print(f"Epoch {epoch+1}/{max_epochs} | Train Loss: {avg_train_loss:.4f} |
Val Loss: {avg_val_loss:.4f}")
            loss_dict['train_loss_list'].append(train_loss)
            loss_dict['val_loss_list'].append(val_loss)

            # 早停逻辑
            if avg_val_loss <= best_loss:
                best_loss = avg_val_loss
                epochs_no_improve = 0
                torch.save(model.state_dict(), save_path)
            else:
                epochs_no_improve += 1
                if epochs_no_improve >= patience:
                    print(f"Early stopping at epoch {epoch+1}")
                    break

        # 加载最佳模型
        model.load_state_dict(torch.load(save_path))
        return model,loss_dict

    def f_B2(X:np.array):
        device = torch.device("cpu")
        # B_model = B_model.to(device)
        B_model.eval()

        X_enum = X[:,0]
        X_num1 = np.array([[0.5]]*X.shape[0])
        X_num2 = X[:,1:]
```

```
        X_num = np.hstack((X_num1,X_num2))
        X_enum_tensor = torch.tensor(X_enum, dtype=torch.long).to(device)
        X_num_tensor = torch.tensor(X_num, dtype=torch.float32).to(device)
        res = B_model(X_num_tensor,X_enum_tensor).detach().numpy()
        return res.reshape(-1,)*1133

    total_rows = X.shape[0]
    # 使用随机索引（推荐）
    backgroud_random_indices = np.random.choice(total_rows, size=500, replace=False)
    explain_random_indices = np.random.choice(total_rows, size=200, replace=False)
    backgroud_np = X[backgroud_random_indices]
    explain_X_np = X[explain_random_indices]   # 选择要解释的样本

    # 创建 SHAP 解释器,test_features 背景
    B_explainer = shap.Explainer(f_B2,backgroud_np)

    plt.figure(figsize=(20, 16), dpi=100)

    # 使用 shap_values_Explanation 绘制摘要图，显示每个特征的影响力

    shap.summary_plot(shap_values_Explanation, explain_X_np,
feature_names=feature_names, plot_type="dot", show=False)

    plt.savefig("./images/SHAP_Summary_Plot2.pdf", format='pdf', bbox_inches='tight')

    plt.show()
```